

TECHNICAL DOCUMENT

ShopSense: AI Shopping Agent

1. System Overview

ShopSense is a **full-stack conversational AI commerce system** that transforms natural language user intent into structured product recommendations and actionable purchase flows.

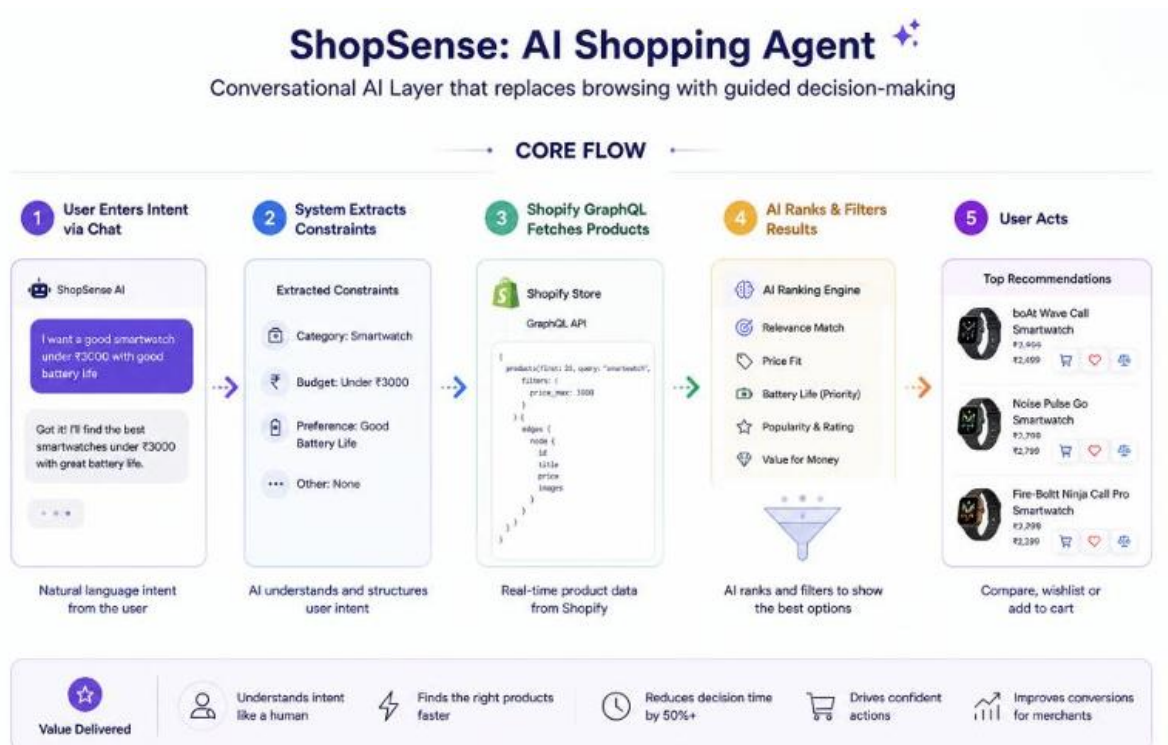
The system replaces traditional browsing workflows with an **agent-driven decision layer** that:

- Interprets user intent
- Narrows product options intelligently
- Explains tradeoffs
- Guides users toward purchase

The system integrates:

- React-based conversational UI
- Node.js backend API
- AI agent layer for reasoning
- Shopify GraphQL for product data
- Firebase Authentication for user management

2. High-Level Architecture



End-to-end architecture from user intent to purchase action

Components

Frontend (React)

- Left: recent chats + quick prompts
- Center: chat + product cards
- Right: live recommendations
- Inline actions:
 - Add to Cart
 - Wishlist
 - Compare

Backend (Node.js API Layer)

- Handles requests from frontend
- Coordinates AI + Shopify calls
- Returns structured responses

AI Agent Layer

- Parses user intent
- Maintains conversational context
- Generates recommendations + reasoning
- Suggests next actions

Shopify GraphQL API

- Fetches real-time product data
- Ensures accuracy and availability

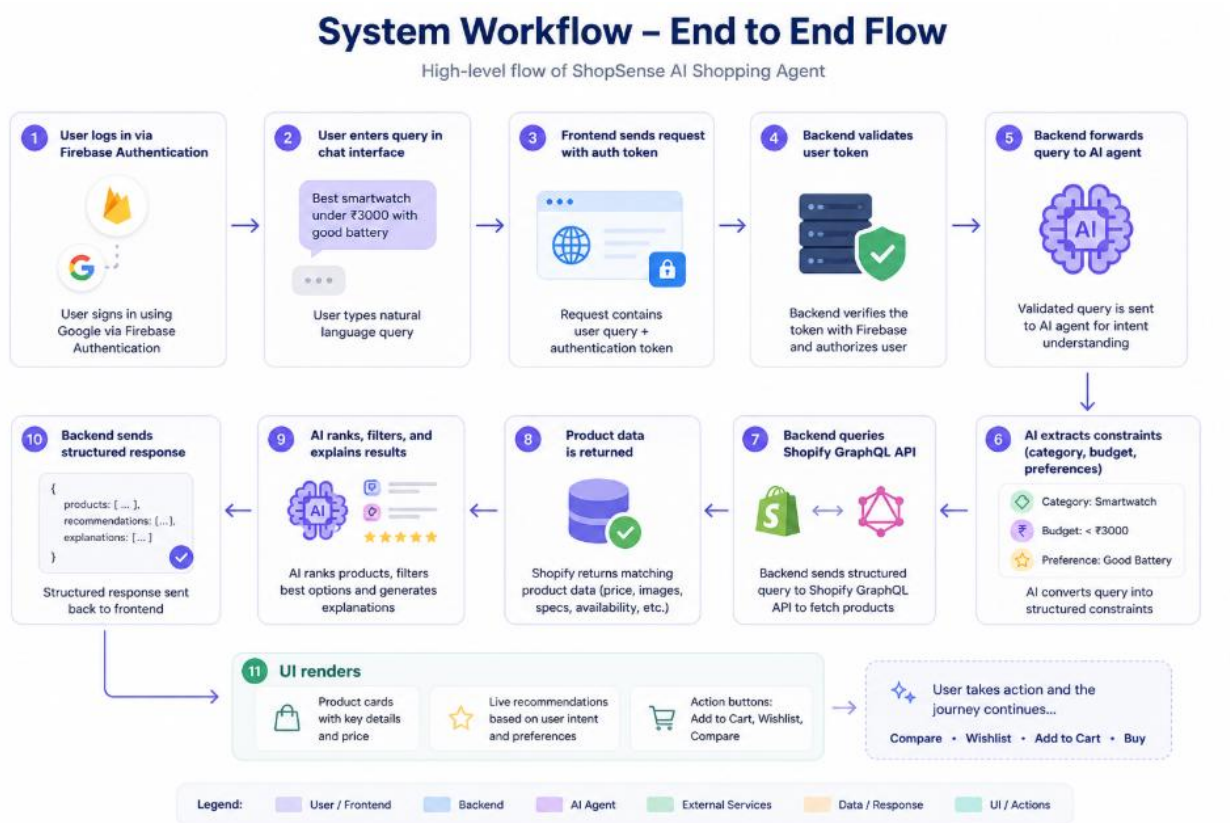
Authentication Layer (Firebase)

- Handles user login via Google Sign-In
- Manages user sessions
- Provides secure identity for personalized interactions

3. System Workflow

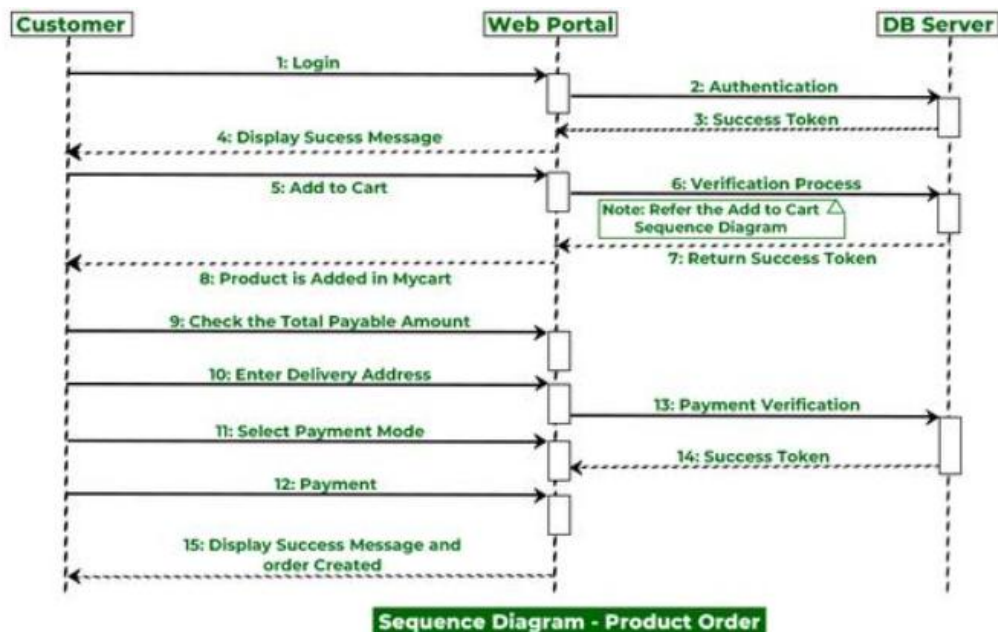
End-to-End Flow

1. User logs in via Firebase Authentication
 2. User enters query in chat interface
 3. Frontend sends request with auth token
 4. Backend validates user token
 5. Backend forwards query to AI agent
 6. AI extracts constraints (category, budget, preferences)
 7. Backend queries Shopify GraphQL API
 8. Product data is returned
 9. AI ranks, filters, and explains results
 10. Backend sends structured response
 11. UI renders:
- Product cards
 - Live recommendations
 - Action buttons (cart, wishlist, compare)



End to End System Workflow

4. Sequence Diagram

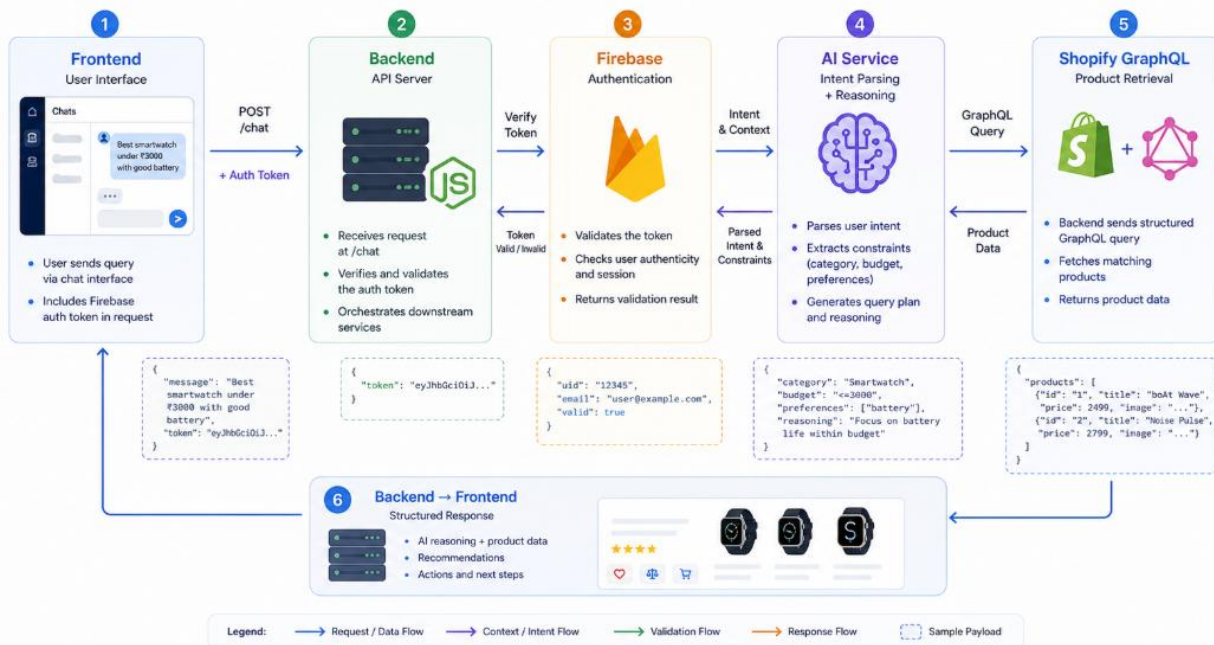


Interaction sequence between user, frontend, backend, AI agent, authentication, and Shopify API

5. API Flow & Data Movement

Flow Summary – ShopSense AI Shopping Agent

High-level data flow across core services



Data flow across frontend, backend, AI services, authentication, and Shopify APIs

6. Shopify GraphQL Integration

Why GraphQL

- Fetches only required fields
- Reduces payload size
- Improves performance

Sample Query

```
products(first: 10, query: "smartwatch") {  
  edges {  
    node {  
      id  
      title  
      priceRange {  
        minVariantPrice { amount }  
      }  
      images(first: 1) {  
        edges { node { src } }  
      }  
    }  
  }  
}
```

7. AI Agent Behavior (Agentic Design)

The system is implemented as a **goal-oriented decision agent**, not a passive chatbot.

Core Capabilities

1. Intent Interpretation

- Converts natural language → structured constraints
- Handles vague or incomplete queries

2. Context Awareness

- Maintains conversational history
- Refines results based on prior inputs

3. Decision Framing

- Narrows large product sets to a small, relevant subset
- Avoids overwhelming users

4. Tradeoff Reasoning

- Explicitly compares alternatives

Example: “This has better battery, but slightly lower display quality.”

5. Action-Oriented Guidance

- Suggests next steps:
 - Compare
 - Add to cart
 - Explore alternatives

6. Proactive Assistance

- Recommends improvements without explicit prompts
- Adapts to user preferences dynamically

8. Real Interaction Examples (Before vs After AI + Fallback Handling)

Scenario 1: Without AI (Traditional Flow)

User Goal:

Purchase a smartwatch under ₹3000

User Behavior

- Searches for product
- Applies filters (price, category)
- Scrolls through large lists
- Opens multiple product pages
- Manually compares features

Issues

- Decision time: ~5–10 minutes
- High cognitive load
- Fragmented comparison process
- No clear recommendation

Scenario 2: With ShopSense AI (Normal Flow)

User Input : “Best smartwatch under ₹3000 with good battery”

System Processing

- AI extracts:
 - Category: Smartwatch
 - Budget: $\leq$ ₹3000
 - Preference: Battery life

AI Output

- Displays 3–4 relevant products
- Provides reasoning:
 - “Best battery in this price range”
- Suggests next step:
 - “Would you like a comparison?”

User Actions

- Compares products
- Adds selected item to cart

Outcome

- Decision time reduced to ~2 minutes
- Reduced number of steps
- Improved decision confidence

Scenario 3: Fallback Handling (AI Failure / API Error)

The system is designed with a **graceful fallback mechanism** to ensure continuity even when AI or external services fail.

Failure Cases Covered

- AI service timeout or error
- Shopify API failure
- Invalid or empty AI response
- Network issues

Fallback Strategy

1. Default Prompt Handling

If AI fails to process input:

- System uses predefined intent templates
- Example mapping:
 - “smartwatch under ₹3000” → category + price filter

2. Cached / Predefined Responses

- Uses stored recommendation templates
- Returns:
 - Popular products
 - Generic category-based suggestions

3. UI Feedback

User sees: “Showing popular options based on your query. You can refine your search.”

4. Minimal Functional Flow Maintained

Even without AI:

- Product list is still displayed
- Add to cart / wishlist still works
- User journey is not blocked

Fallback Example

User Input:

“Best smartwatch under ₹3000”

Fallback Output:

- Displays filtered product list
- No advanced reasoning
- Basic sorting (price or popularity)

Result

- System remains usable
- No hard failure or blank state
- Maintains core shopping functionality

Summary of Behavior Modes

Mode	Capability
AI Active	Intelligent recommendations + reasoning
AI Partial	Limited reasoning + structured results
AI Failure	Default filtering + predefined suggestions

Key Insight

The system is designed to **degrade gracefully**, ensuring that core shopping functionality remains available even when AI components fail.

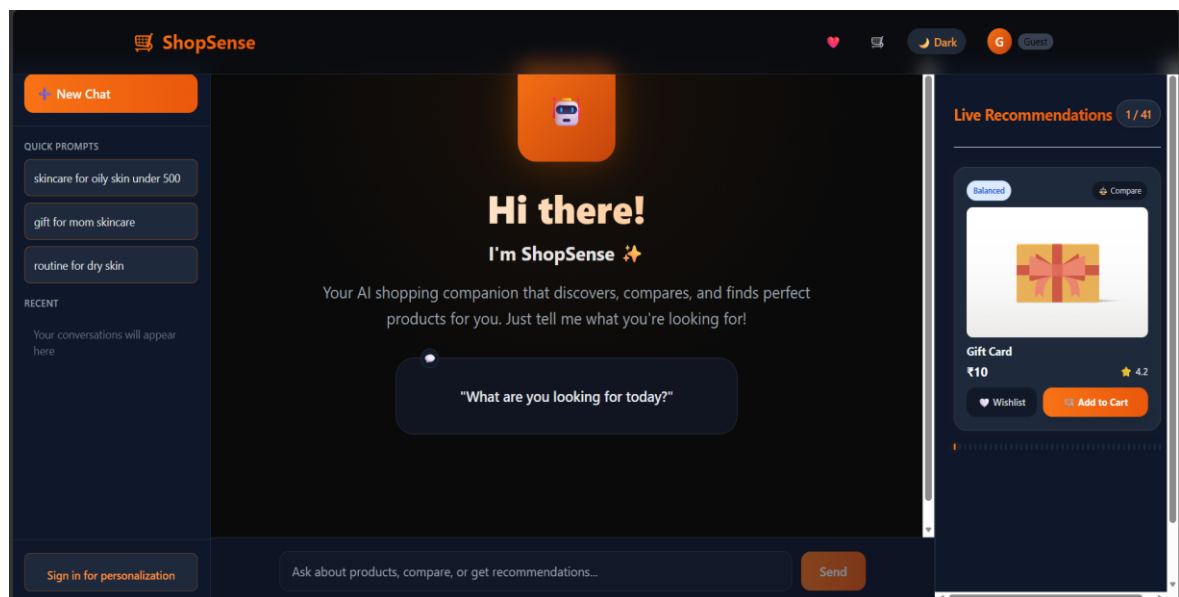
9. UI Architecture

Three-Panel Layout

- **Left:** Recent chats + quick prompts
- **Center:** Chat + product cards
- **Right:** Live recommendations

Components

- Chat.jsx
- ProductCard.jsx
- Comparison.jsx
- ChatRightSidebar.jsx
- ChatLeftSidebar.jsx



10. Features Implemented

- Conversational search
- AI-based recommendations
- Real-time Shopify integration
- Product comparison
- Wishlist functionality
- Add-to-cart system
- Live recommendation panel
- Firebase authentication
- Session Memory Management

11. Performance Considerations

- GraphQL reduces data payload
- Async processing improves responsiveness
- Target latency: <2 seconds

12. Limitations

- No deep personalization yet
- Limited to Shopify ecosystem
- Rule-based ranking system

13. Future Technical Improvements

- Personalization engine
- Vector search (semantic retrieval)
- Multi-store aggregation
- Redis caching
- Fine-tuned recommendation models

14. Authentication & User Management

Technology

- Firebase Authentication (Google Sign-In)

Flow

1. User signs in via Google
2. Firebase returns authentication token
3. Frontend stores session
4. Token included in API requests
5. Backend validates token

15. Deployment

- Vercel
- APIs:
 - Shopify GraphQL
 - AI service (Groqai)
 - Firebase Auth

16. Fallback Handling Across System

The system is designed to **handle failures gracefully at every stage**, ensuring that the user experience remains functional even when individual components fail.

1. Authentication (Firebase Failure)

Failure Cases

- Firebase service unavailable
- Token validation failure
- User not authenticated

Fallback Handling

- Allow **limited guest mode** (optional, if implemented)
- Restrict sensitive actions (e.g., add to cart persistence)
- Prompt user:

“Please sign in to continue full functionality.”

System Behavior

- Chat and product discovery remain available
- Session-dependent features are disabled

2. Frontend → Backend Communication Failure

Failure Cases

- Network issues
- API timeout
- Server unreachable

Fallback Handling

- Retry mechanism (basic retry logic)
- Show UI message:

“Connection issue. Please try again.”

System Behavior

- Prevents blank UI
- Maintains user awareness

3. AI Service Failure

Failure Cases

- AI API timeout
- Invalid response
- Parsing failure

Fallback Handling

A. Rule-Based Intent Parsing

- Extract keywords manually:
 - Category
 - Price
- Example:
 - “smartwatch under ₹3000” → filter query

B. Predefined Prompt Templates

- Use stored responses:
 - “Here are some popular options in your range.”

C. Reduced Functionality Mode

- No reasoning explanations
- Basic filtering + sorting applied

System Behavior

- Maintains product discovery
- Avoids blocking user flow

4. Shopify API Failure

Failure Cases

- API downtime
- Empty response
- Rate limiting

Fallback Handling

A. Cached Results (if available)

- Serve previously fetched products

B. Default Product Lists

- Show:
 - Trending products
 - Popular items

C. UI Notification

“Live data unavailable. Showing popular products.”

System Behavior

- Keeps UI populated
- Avoids empty states

5. Data Processing / Ranking Failure

Failure Cases

- AI ranking failure
- Missing product attributes

Fallback Handling

- Sort by:
 - Price
 - Popularity

System Behavior

- Provides basic but usable results
- Removes dependency on AI reasoning

6. UI Rendering Failure

Failure Cases

- Component crash
- Missing data fields

Fallback Handling

- Show placeholder UI:
 - Default image
 - “Data unavailable” labels

System Behavior

- Prevents UI break
- Maintains visual consistency

7. Action Failures (Cart / Wishlist)

Failure Cases

- API failure
- Auth issue

Fallback Handling

- Show message:
“Action failed. Please try again.”
- Retry option

System Behavior

- Prevents silent failures
- Keeps user informed

8. Overall Fallback Strategy

The system follows a **graceful degradation model**:

Level	Behavior
Full System	AI + real-time data + actions
Partial Failure	Limited AI + real-time data
Major Failure	Static data + basic filtering
Critical Failure	Inform user + retry

Key Principle

The system never reaches a “dead end” — it always provides a usable fallback state.

18. Conclusion

ShopSense demonstrates a **complete, production-oriented AI commerce system** that goes beyond basic chatbot functionality.

It combines:

- Conversational AI for intent understanding
- Structured data retrieval via Shopify GraphQL
- Secure user authentication with Firebase
- Interactive UI for decision-making
- Robust fallback handling for reliability

Core Outcome

The system successfully:

- Replaces traditional browsing workflows
- Reduces decision time
- Improves user confidence
- Maintains functionality even under failure conditions

Final Insight

ShopSense is not just an AI feature—it is a **decision engine** that transforms e-commerce from a navigation-heavy process into a guided, intelligent experience, while ensuring reliability through robust fallback mechanisms.